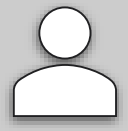
By: Bilal Ahmed

Exception handling in Java

Exception

Abnormal condition in a code sequence, which arises at run-time

Run Time Error



Disrupts the normal flow of program

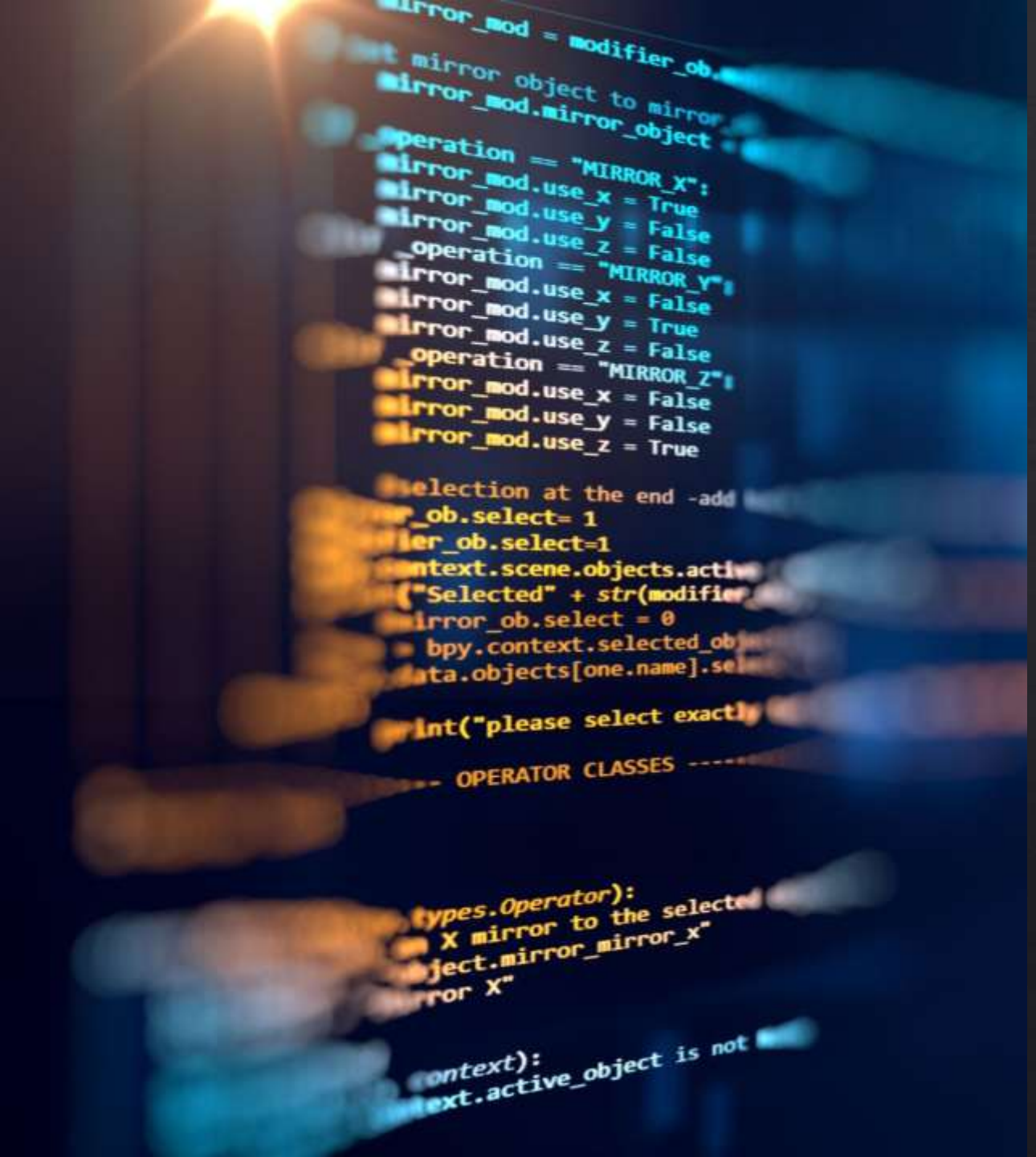
Example situations

- Dividing by zero
- Accessing invalid array index
- File not found

Code Example

- Crash → ArithmeticException

```
int a = 10;
int b = 0;
System.out.println(a / b);
```



Why Exception Handling?

- Without handling:
  - Program stops ❌
  - Bad user experience ❌
- With handling:
  - Program continues ✅
  - Errors are managed properly ✅

Java Exceptions

When an error occurs, Java will normally stop and generate an error message.

The technical term for this is: Java will throw an **exception** (throw an error).

Basic Structure

try → “I’ll try this code”

catch → “If something goes wrong, handle it”

```
try {
// risky code
} catch (ExceptionType e) {
// handling code
}
```

What will be the output of the following code?

```
public class Main {
    public static void main(String[] args) {
      int a = 10;
      int b = 0;
      int result = a / b;
      System.out.println(result);
      System.out.println("Program after error...");
    }
}
```

Let’s handle it gracefully

```
public class Main {
    public static void main(String[] args) {
      int a = 10;
      int b = 0;

      try{
        int result = a / b;
        System.out.println(result);
      }

      catch (ArithmeticException e){
        System.out.println("Can't divide a number by zero");
      }

      System.out.println("Continued Program after
error…");
    }
}
```

Variable in catch

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int result = a / b;  // will cause exception
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Error message: " + e.getMessage());
            System.out.println("Full details:");
            e.printStackTrace(System.out);
        }

        System.out.println("Program continues...");
    }
}
```

What will be the output of the following?

```
public class Main {
    public static void main(String[] args) {
      int arr[] =  {1,2,3,4,5};

      System.out.println(arr[2]);
      try{
        System.out.println(arr[8]);
      }

      catch (ArithmeticException e){
        System.out.println("Index out of bound error");
      }

      System.out.println("Continued Program after
error…");
    }
}
```

Solution

```
public class Main {
    public static void main(String[] args) {
      int arr[] =  {1,2,3,4,5};

      System.out.println(arr[2]);
      try{
        System.out.println(arr[8]);
      }

      catch (ArrayIndexOutOfBoundsException e){
        System.out.println("Index out of bound error");
      }

      System.out.println("Continued Program after error…");
    }
}
```

Common Java Exceptions

- **ArithmeticException** → Division or invalid math operation
- **NullPointerException** → Using a null (non-existing) object
- **ArrayIndexOutOfBoundsException** → Accessing array outside its limit
- **NumberFormatException** → Invalid string-to-number conversion
- **IOException** → Error during file input/output

Handling Multiple Exceptions

```
public class Main {
    public static void main(String[] args) {
        try {
            int[] arr = {1, 2, 3};
            System.out.println(arr[5]); // Index Error
}
        catch (ArithmeticException e) {
            System.out.println("Math error!");
        }  catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of range!");
        }
    }
}
```

Handling Multiple Exceptions

```
public class Main {
    public static void main(String[] args) {

        try {
            int[] arr = {1, 2, 3};
            System.out.println(arr[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index out of range!");
        }

        try {
            int k = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Math error!");
        }
    }
}
```

Finally Block

- The finally block is used to **guarantee execution of important code**, no matter what happens.
- **Simple idea:**
  - “Run this code whether error occurs or not”
- **When is it useful?**
  - Closing files 📄
  - Releasing resources 🔌
  - Closing database connections
- These must happen **even if an exception occurs**

Finally Block

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            System.out.println(a / b);  // causes exception
        }
        catch (ArithmeticException e) {
            System.out.println("Error occurred!");
        }
        finally {
            System.out.println("Cleaning up resources (finally block)");
        }

        System.out.println("Program continues…");
    }
}
```

Key Insight

- try → risky code
- catch → handle error
- finally → **always runs (important work)**
- You open a file
- Even if error happens, you must **close it**

Throw keyword

- Used to **manually create an exception**
- Used when you want to enforce rules in your class

```
public class Main {

    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Not
allowed: Age is less than 18");
        } else {
            System.out.println("Access granted");
        }
    }

    public static void main(String[] args) {
        checkAge(16);
    }
}
```

Throws Keyword

- Used to **inform caller that method may throw exception**
- **We did NOT use throw**
- **But Java methods like FileReader may cause errors internally**
- **So we use throws IOException to pass responsibility**

```
import java.io.*;

public class Main {

    static void readData() throws IOException {
        FileReader file = new FileReader("test.txt");
        file.read();
        file.close();
    }

    public static void main(String[] args) {
        try {
            readData();
        } catch (IOException e) {
            System.out.println("File error occurred");
        }
    }
}
```

In a nutshell

- throw = "I am throwing error”
- throws = "I may throw error“
- throw → used only when you manually force an error
- throws → used even if exception is automatic

Throw and throws together

throws = warning
throw = action

```
public class Main {
    // declares possible exception
    static void checkMarks(int marks) throws Exception {

        // manually creating exception
        if (marks < 0 || marks > 100) {
            throw new Exception("Invalid marks entered!");
        }

        System.out.println("Valid marks: " + marks);
    }

    public static void main(String[] args) {

        try {
            checkMarks(120);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        System.out.println("Program continues…");
    }
}
```

Real-life Example 01 – Banking System

```
class BankAccount {
    int balance = 1000;

    void withdraw(int amount) {
        try {
            if (amount > balance) {
                throw new Exception("Insufficient balance!");
            }
            balance -= amount;
            System.out.println("Withdraw successful. Remaining balance: " + balance);
        }
        catch (Exception e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Take your card");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();

        acc.withdraw(500);
        System.out.println("-----------------");
        acc.withdraw(1500);
    }
}
```

Real-life Example 02 – Shopping Cart System

```
class Cart {

    void checkout(int items) {
        try {
            if (items <= 0) {
                throw new Exception("Cart is empty!");
            }
            System.out.println("Order placed for " + items + " items");
        }
        catch (Exception e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Session closed, logout user");
        }
    }

    public static void main(String[] args) {
        Cart c = new Cart();
        c.checkout(0);
    }
}
```

Real-life Example 03 – Shopping Cart System

```
class Bank {

    int balance = 500;

    void transfer(int amount) {
        try {
            if (amount > balance) {
                throw new ArithmeticException("Not enough balance!");
            }
            balance -= amount;
            System.out.println("Transfer successful");
        }
        catch (ArithmeticException e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Transaction completed");
        }
    }

    public static void main(String[] args) {
        Bank b = new Bank();
        b.transfer(500);
        b.transfer(1000);
    }
}
```

Summary

Concept	Meaning
throw	Something went wrong right now
throws	This operation is risky
try-catch	Handle problem safely
finally	Always cleanup / close session

Big Picture

- **predict errors (throws)**
- **create errors when needed (throw)**
- **handle safely (try-catch)**
- **cleanup always (finally)**